

Mundo dos Blocos de Tamanho Variável

Fundamentos de Inteligência Artificial – UFAM / ICOMP

Camila dos Santos Soares; Daniel Nunes Santos; Fernando Reis; Nicolay Lima de Souza;
Pedro Matheus Melo Pereira; Pedro Vinícius Diaz de Alencar

1 Introdução ao Problema

O objetivo é mover blocos de comprimentos diferentes sem colisões e com apoio suficiente. Resolvemos a Situação 1 até Sf4, a Situação 2 até S5 e a Situação 3 até S7. Primeiro definimos o domínio e os planos manuais; depois usamos Python e MiniSat 2.2 para gerar planos a partir de cláusulas CNF.

2 Descrição Formal do Mundo dos Blocos de Tamanho Variado

Domínio em LPO. Os blocos são $B = \{a, b, c, d\}$, com comprimentos $\ell(a) = 1$, $\ell(b) = 1$, $\ell(c) = 2$ e $\ell(d) = 3$. A mesa é T . Existem sete pontos, de 0 a 6, e seis slots $s_i = [i, i + 1]$, para $i = 0, \dots, 5$. Um bloco em p cobre os slots de p até $p + \ell(b) - 1$. A posição deve satisfazer $0 \leq p \leq 6 - \ell(b)$; os níveis vão de 0 a 3, sendo 0 a mesa. Encostar pelas bordas não significa compartilhar um slot.

Predicado	Significado no instante t
$at(b, p, t)$	O bloco b começa no ponto p .
$lev(b, l, t)$	O bloco b está no nível l .
$clr(b, t)$	Nenhum bloco mais alto cobre slots de b .
$on(b, y, t)$	b tem apoio em y ; pode haver mais de um apoio.

Cada bloco possui exatamente uma posição e um nível. Usamos a relação auxiliar de cobertura:

$$cov(b, s, l, t) \leftrightarrow lev(b, l, t) \wedge \exists p [at(b, p, t) \wedge p \leq s < p + \ell(b)].$$

Para blocos, $on(b, y, t)$ vale quando estão em níveis consecutivos e compartilham pelo menos um slot; $on(b, T, t) \leftrightarrow lev(b, 0, t)$. Já $clr(b, t)$ vale quando não existe outro bloco em nível maior cobrindo algum slot de b . Essas condições se aplicam a todos os blocos e instantes do domínio.

Estabilidade. Para todo bloco acima da mesa, pelo menos $\lceil \ell(b)/2 \rceil$ slots sob ele precisam estar ocupados no nível imediatamente inferior. Assim, a e b precisam de 1 slot, c de 1 e d de 2. Conta-se o conjunto de slots apoiados: d pode formar uma ponte sobre a e b com um vão no meio.

Ação. $move(b, y, p, t)$ coloca b sobre y a partir de p . Exige b livre, posição válida, destino diferente do atual, espaço no destino e acima dele, além de estabilidade após o movimento. Se y é bloco, deve haver sobreposição horizontal e o novo nível é $nivel(y) + 1$; para a mesa, é 0. Uma ação ocorre por passo, sem transportar outros blocos.

Para reproduzir os blocos lado a lado nas figuras, substituímos $clr(y)$ do manual por espaço livre na região utilizada do apoio. O bloco movido continua exigindo clr e não pode entrar sob outro bloco, inclusive no vão de uma ponte.

Efeito	O que acontece
Adds	Nova posição, novo nível e novos apoios; antigos apoios ficam livres se nada permanecer acima.
Deletes	Posição e nível antigos quando diferentes, apoios perdidos e <i>clr</i> dos apoios agora cobertos.
Permanência	Posição e nível dos demais blocos; comprimentos, mesa, pontos e slots não mudam. <i>clr</i> e <i>on</i> acompanham a geometria.

Ordem parcial. $A - P \rightarrow B$ significa que A estabelece uma condição P necessária a B, preservada entre as ações. Chamando os quatro movimentos manuais da Situação 1 de A1 a A4, temos:

A1 -- *clr(a)* --> A2
A2 -- *slot s3 livre na mesa* --> A3
A3 -- *clr(c)* --> A4

Movimentos sem dependência podem trocar de ordem. As pré-condições e a permanência representam essas dependências; o SAT produz uma sequência compatível, sem receber o plano manual como restrição.

Por exemplo, na Situação 2 automática, após colocar *c* sobre *d*, os movimentos finais de *a* e *b* podem trocar de ordem, pois usam slots distintos de *c*.

3 Codificação CNF

Substituímos as variáveis da LPO pelos blocos, posições, níveis e instantes possíveis. Cada átomo vira uma variável booleana numerada. Usamos *at*, *lev*, *clr*, *move* e duas auxiliares: *pos(b,p,l,t)*, que reúne posição e nível, e *occ(s,l,t)*, que indica slot ocupado. *on* é reconstruído, sem variável própria. Os horizontes são $H = 4, 5, 6$: estados de 0 a H e ações de 0 a H-1.

Grupo de cláusulas	Regra codificada
Inicial e meta	Fixam <i>at</i> e <i>lev</i> nos extremos do plano.
Unicidade e colisões	Uma posição e um nível por bloco; no máximo um ocupante por célula.
Estabilidade e clear	Apoio mínimo abaixo; <i>clr</i> equivale à ausência de ocupação acima.
Pré-condições e efeitos	Bloco livre, apoio compatível, destino livre, nova posição e novo nível; sem ação vazia.
Frame axioms	Sem movimento de <i>b</i> , seus <i>at</i> e <i>lev</i> permanecem. A definição de <i>clr</i> preserva seu valor enquanto a ocupação acima não mudar.
Ação única	Exatamente um <i>move</i> por instante.

Exemplos de conversão: $move \rightarrow clr$ vira $\neg move \vee clr$. A equivalência $pos \leftrightarrow (at \wedge lev)$ é representada por três cláusulas:

$$(\neg pos \vee at) \wedge (\neg pos \vee lev) \wedge (\neg at \vee \neg lev \vee pos).$$

Para *d*, os três slots de apoio *x, y, z* exigem $(x \vee y) \wedge (x \vee z) \wedge (y \vee z)$, condicionado à posição de *d*. Isso garante pelo menos dois slots ocupados. No DIMACS, os IDs representam variáveis, o sinal negativo indica negação e cada cláusula termina em 0.

4 Exemplos dos 3 Cenários Codificados para CNF em LP

Cada par (p, l) representa $at(bloco, p, t) \wedge lev(bloco, l, t)$. Todos os pares de uma linha valem juntos: inicialmente em $t = 0$ e na meta em $t = H$. As linhas fixam os átomos correspondentes com cláusulas unitárias.

Situação / estado	a	b	c	d
1 e 3: S0	(3,0)	(5,0)	(0,0)	(3,1)
1: Sf4	(0,1)	(5,0)	(0,0)	(2,0)
2: S0	(0,1)	(1,1)	(0,0)	(3,0)
2: S5	(4,2)	(5,2)	(4,1)	(3,0)
3: S7	(0,1)	(1,1)	(0,0)	(3,0)

Planos manuais. Na tabela, $move(b, y, p)$ omite apenas o instante, igual ao passo menos 1. O par após a seta é a nova posição e nível do bloco movido; os demais permanecem iguais. Assim, cada linha permite obter o próximo estado até a meta.

Passo	Situação 1	Situação 2	Situação 3
1	$move(d, c, 0) \rightarrow (0, 1)$	$move(b, T, 2) \rightarrow (2, 0)$	$move(d, c, 0) \rightarrow (0, 1)$
2	$move(a, b, 5) \rightarrow (5, 1)$	$move(a, b, 2) \rightarrow (2, 1)$	$move(a, b, 5) \rightarrow (5, 1)$
3	$move(d, T, 2) \rightarrow (2, 0)$	$move(c, d, 4) \rightarrow (4, 1)$	$move(d, T, 2) \rightarrow (2, 0)$
4	$move(a, c, 0) \rightarrow (0, 1)$	$move(a, c, 4) \rightarrow (4, 2)$	$move(a, c, 0) \rightarrow (0, 1)$
5	–	$move(b, c, 5) \rightarrow (5, 2)$	$move(b, c, 1) \rightarrow (1, 1)$
6	–	–	$move(d, T, 3) \rightarrow (3, 0)$

Nas Situações 1 e 3, a vai temporariamente sobre b : deixá-lo em $p = 4$ na mesa, como no exemplo do manual, impediria d de ocupar $[2, 5]$. Na Situação 3, S5 e S6 desenham a mesma configuração; não é necessário um movimento vazio.

Comparação com as execuções reais.

Situação	Manual / solver	Comparação
1	4 / 4 ações	Mesma sequência, chegando a Sf4.
2	5 / 5 ações	Outro apoio temporário para a e inversão dos dois últimos movimentos; mesmo S5.
3	6 / 6 ações	Mesma sequência, chegando a S7.

Na Situação 2, a sequência do solver foi: $move(b, T, 2)$; $move(a, d, 3)$; $move(c, d, 4)$; $move(b, c, 5)$; $move(a, c, 4)$. Os três resultados são SAT. Os horizontes vêm dos planos manuais; não se afirma optimalidade.

No início das Situações 1 e 3, $on(d, a, 0)$ e $on(d, b, 0)$ descrevem a ponte. Em Sf4, a fica sobre c ; em S5, a e b ficam sobre c , e c sobre d ; em S7, a e b ficam sobre c . Os blocos restantes ficam na mesa.

5 Mapeamento: Descrição Formal para Código

Conceito	Parte do código
Blocos, comprimentos e cenários	BLOCKS, MAX_POINT, MAX_LEVEL, CENARIOS em <code>bw2cnf_var.py</code>
Posições, slots e sobreposição	<code>posicoes</code> , <code>slots</code> , <code>sobrepoe</code>
Predicados, ações e regras CNF	<code>gerar_cnf</code> ; <code>var</code> numera os símbolos
DIMACS e MAP	<code>salvar</code>
MiniSat 2.2 e resposta numérica	<code>resolver</code>
Plano legível e apoios	<code>interpretar</code> e <code>derivar_on</code> , em <code>interpretar.py</code>

6 Execução Passo a Passo: Gerar o CNF, Mapeamento e Execução

No Linux, abra o terminal dentro de `TrabalhoIA_MundoBlocos`. Prepare o ambiente uma vez:

```
python3 -m venv ../.venv_blocos
source ../.venv_blocos/bin/activate
python3 -m pip install -r requirements.txt
```

Em Ubuntu/Mint, se faltar `venv`, instale `python3-venv` com o gerenciador de pacotes. O ambiente fica fora da pasta do trabalho. Gere CNF e MAP, execute MiniSat e interprete os três resultados:

```
for i in 1 2 3; do
    python3 bw2cnf_var.py "$i" --resolver
    python3 -B interpretar.py "situacao$i/resultado$i.txt"
done
```

O argumento 1, 2 ou 3 seleciona a situação. Sem `-resolver`, o primeiro comando apenas gera CNF e MAP. Com essa opção, lê o CNF salvo e executa MiniSat 2.2 pela biblioteca PySAT, sem instalar outro executável. Cada pasta `situacaoN` recebe seu CNF, MAP e `resultadoN.txt`. A opção `-B` evita cache do Python. Em um novo terminal, reative o ambiente com `source ../.venv_blocos/bin/activate`.

7 Interpretação da Saída do SAT Solver

`resultadoN.txt` contém SAT e o modelo real do solver: números positivos são variáveis verdadeiras e negativos são falsas. UNSAT indicaria ausência de plano no horizonte usado. O MAP associa cada número a um símbolo, por exemplo, `move(d,c,0,0)`.

`interpretar.py` lê o MAP da mesma pasta, seleciona as ações verdadeiras e ordena por t . Exemplo: $t=0$: mover 'd' para cima de 'c', no ponto $p=0$. Depois mostra posições e níveis finais e reconstrói *on*: nível 0 indica mesa; nos demais níveis, procura todos os blocos no nível imediatamente inferior com slots compartilhados. Assim, reconhece um apoio simples ou uma ponte. Não se deve misturar o resultado de uma situação com o MAP de outra.